

Artificial Intelligence

AI2002

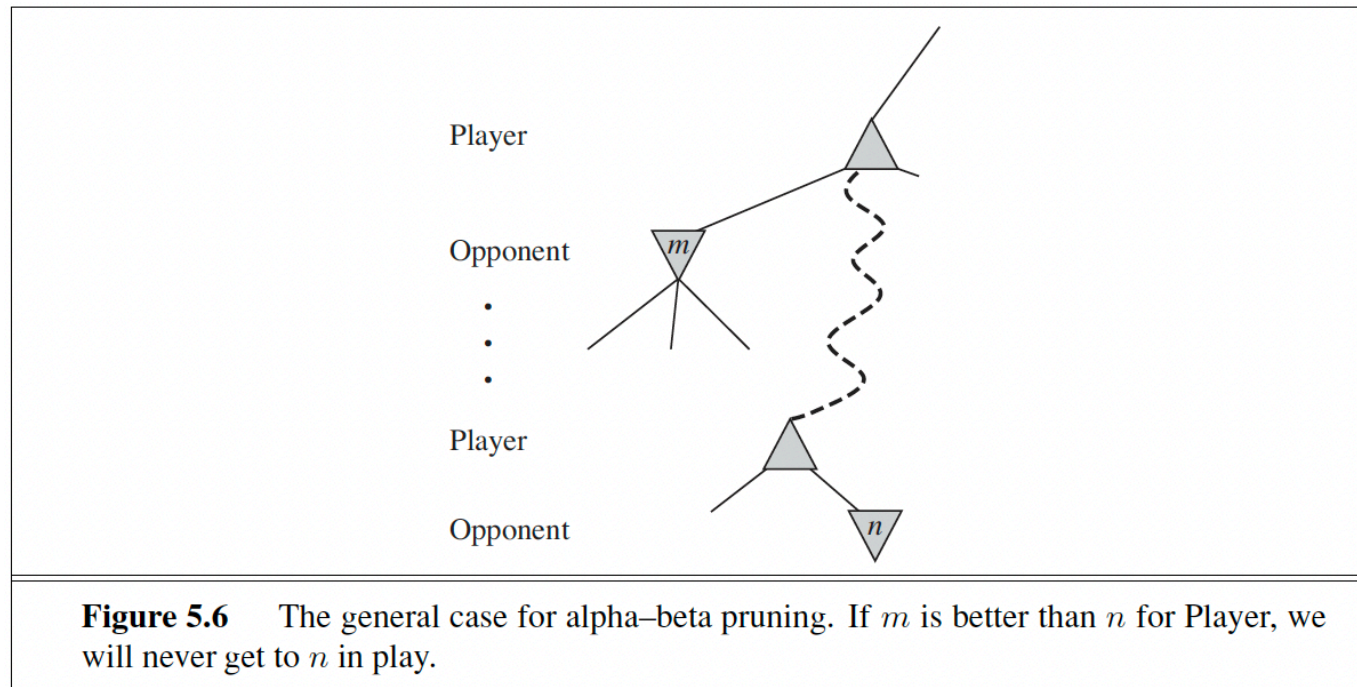
Rushda Muneer

Spring 2026

Alpha-Beta Pruning

- The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree.
- Unfortunately, we can't eliminate the exponent, but it turns out we can effectively cut it in half.
- The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree.
- We use the idea of **pruning** to eliminate large parts of the tree from consideration.
- The particular technique we examine is called **alpha-beta pruning**.
- When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.

Alpha – Beta Pruning



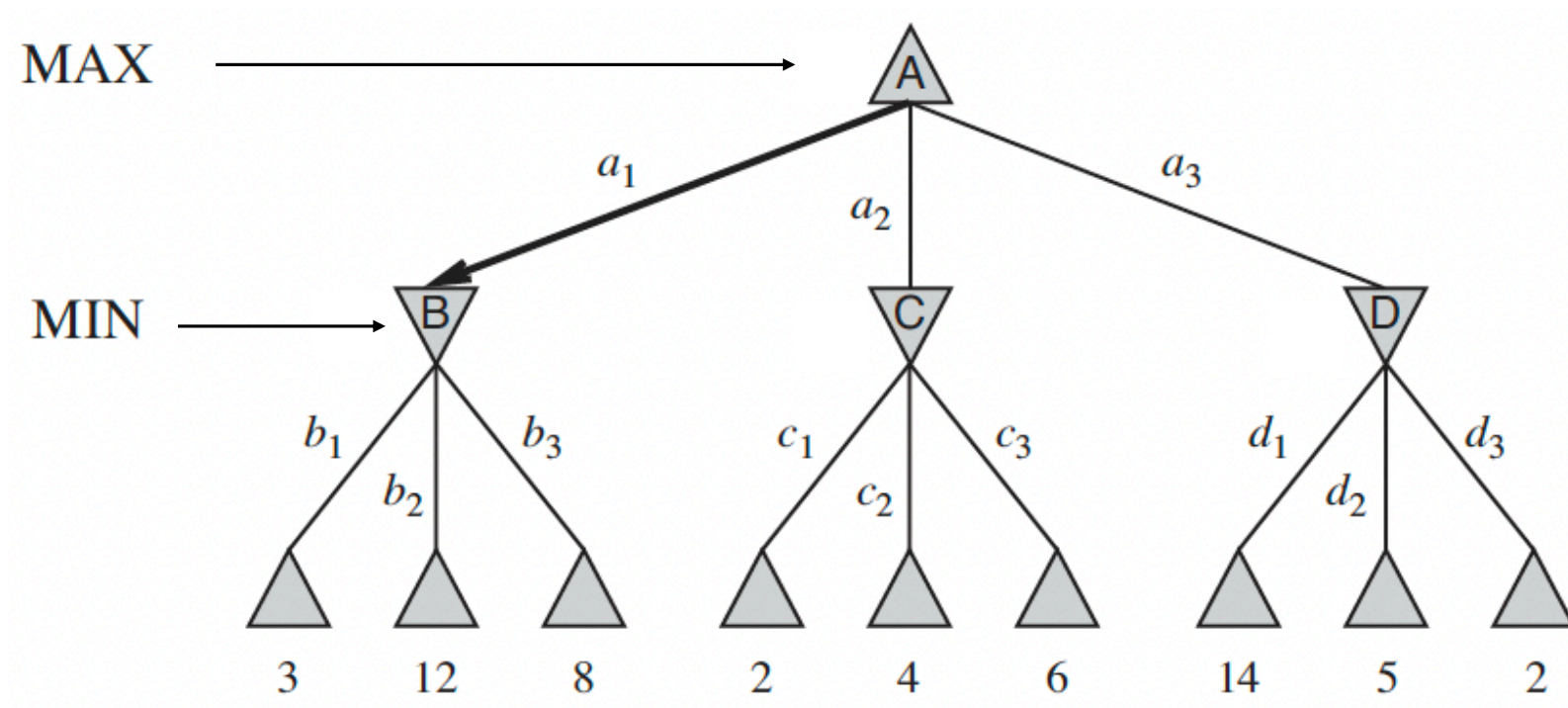
If **Player** has a **better choice m** either at the parent node of n or at any choice point further up, then **n will never be reached in actual play**. So once we have found out enough about n (by examining some of its descendants) to reach this conclusion, we can prune it.

Alpha – Beta Pruning

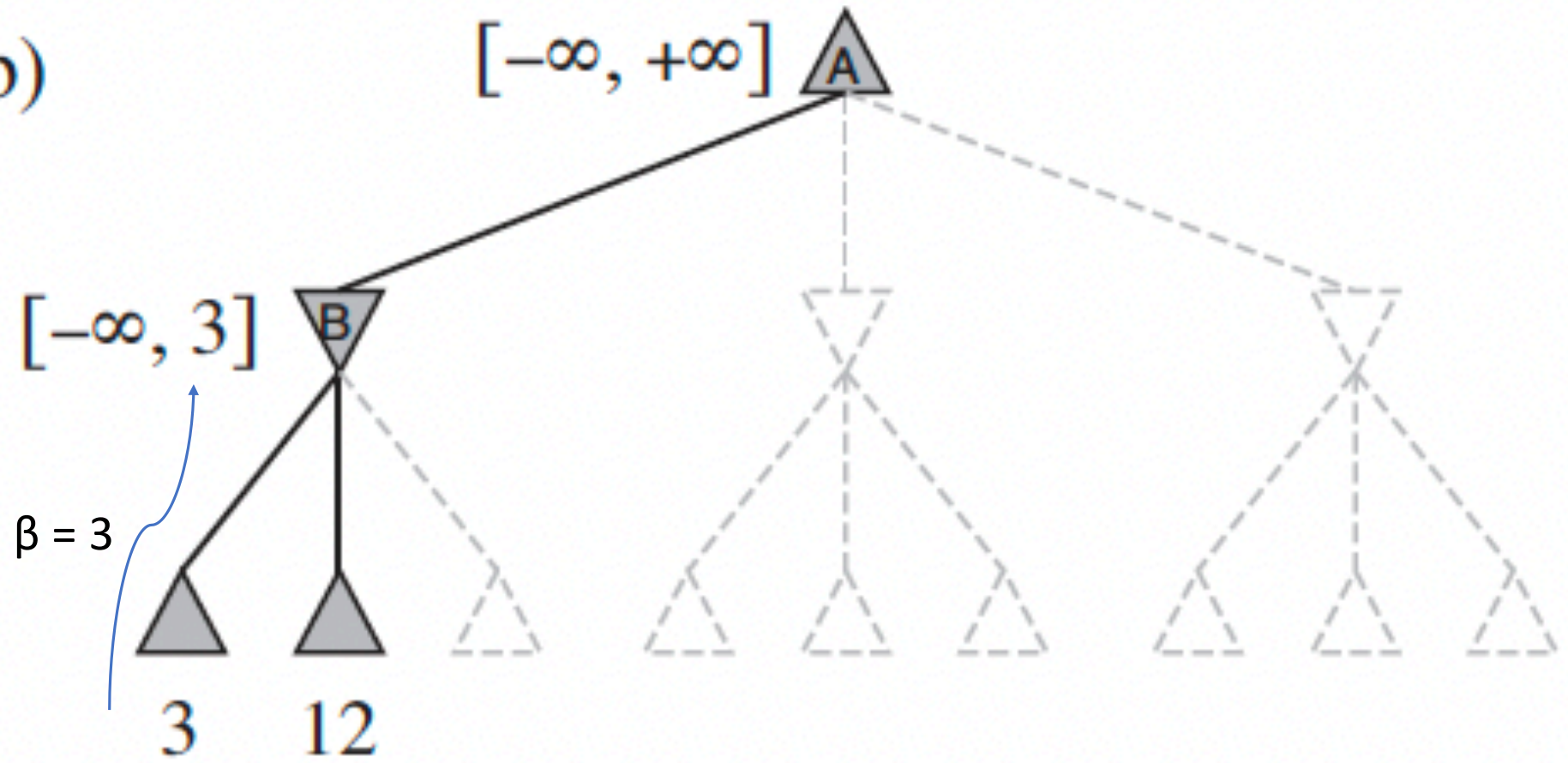
- The minimax search is **depth-first**, so at any one time we just have to consider the nodes along a **single path in the tree**.
- Alpha–beta pruning gets its name from the following **two parameters** that **describe bounds** on the backed-up values that appear anywhere along the path:
- α = the value of the **best** (i.e., highest-value) **choice** we have found so far at any choice point along the path **for MAX**.
- β = the value of the **best** (i.e., lowest-value) **choice** we have found so far at any choice point along the path **for MIN**.
- Alpha–beta search **updates the values of α and β** as it goes along and prunes the remaining branches at a node as soon as the value of the **current node** is known to be **worse than the current α or β value for MAX or MIN**, respectively.

Alpha – Beta Pruning

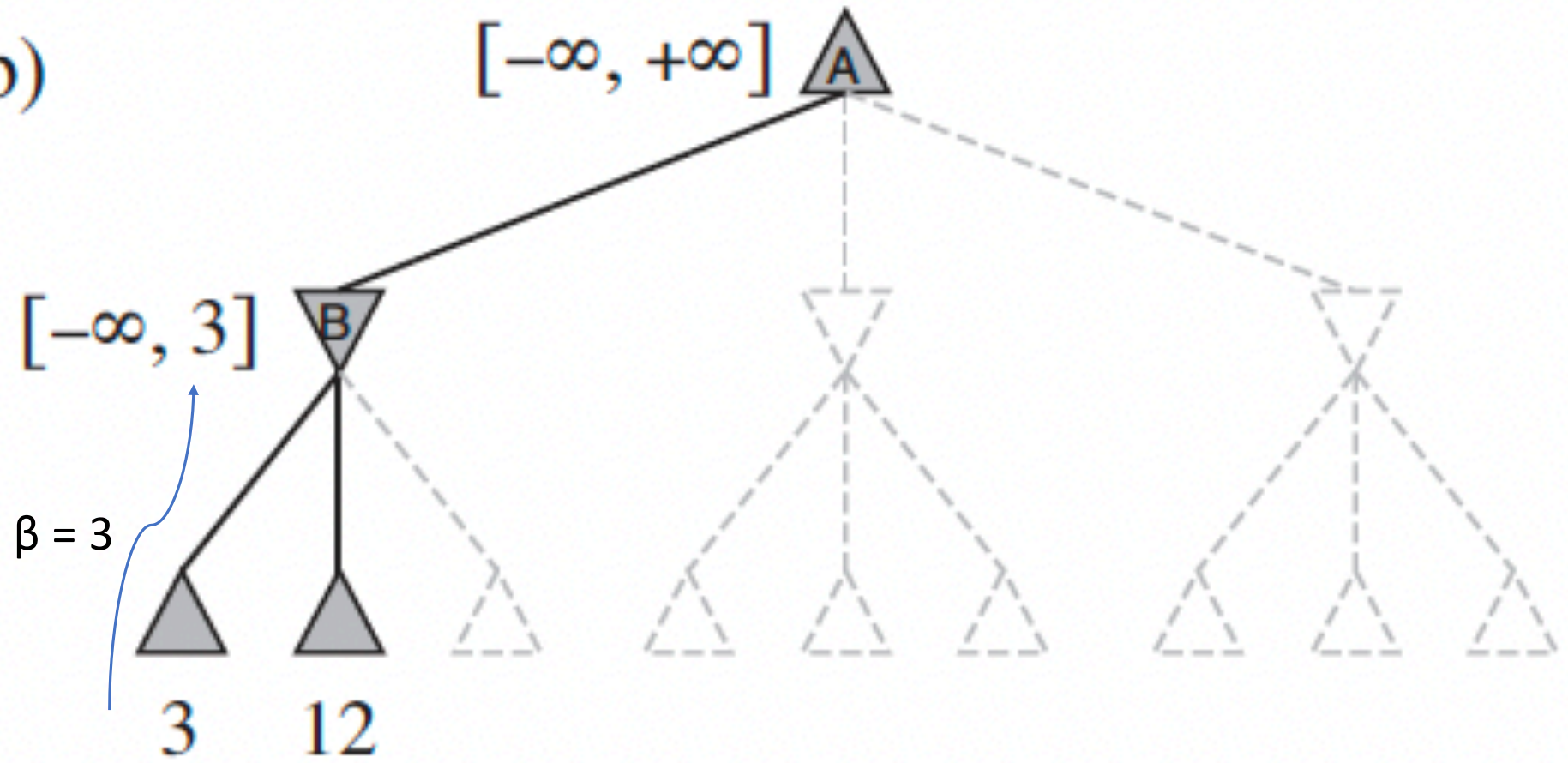
- The **alpha value**, associated with MAX nodes, can never **decrease**, and the **beta value**, associated with MIN nodes, can never **increase**.
- Alpha is the **lower bound** of the maximizing nodes
- Beta is the **upper bound** of the minimizing nodes
- At **minimizing nodes**, we **stop evaluating** children if we get a child whose **value is less than the current lower bound** (alpha)
- At maximizing nodes, we **stop evaluating** children if we get a child whose **value is greater than the current upper bound** (beta)
- At the root of the search tree, **alpha** and **beta** are set to $-\infty$ and $+\infty$ respectively.
- Maximizing and minimizing nodes update **alpha** and **beta** values respectively
- If **alpha > beta**, we stop evaluating the children.



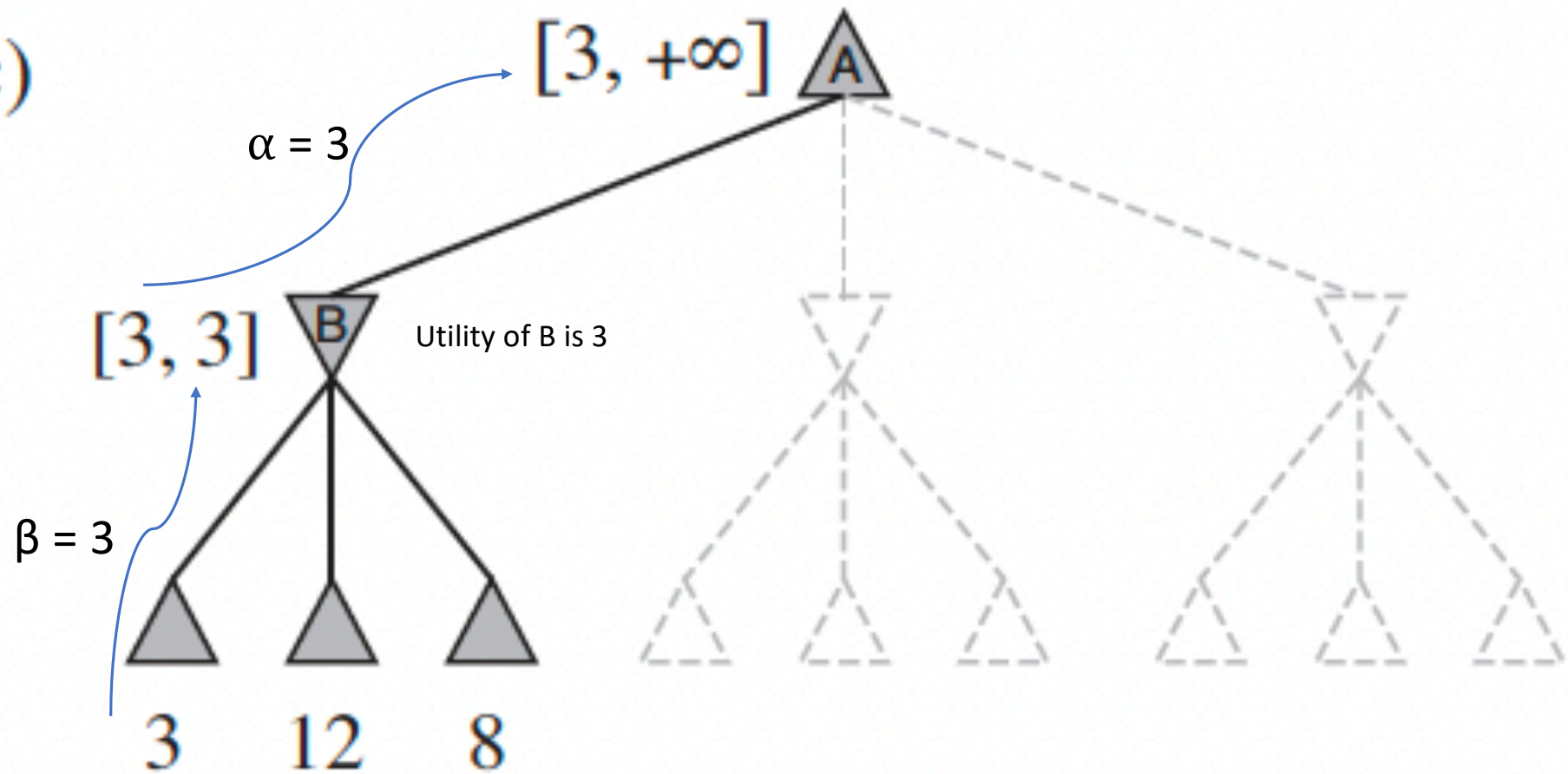
(b)



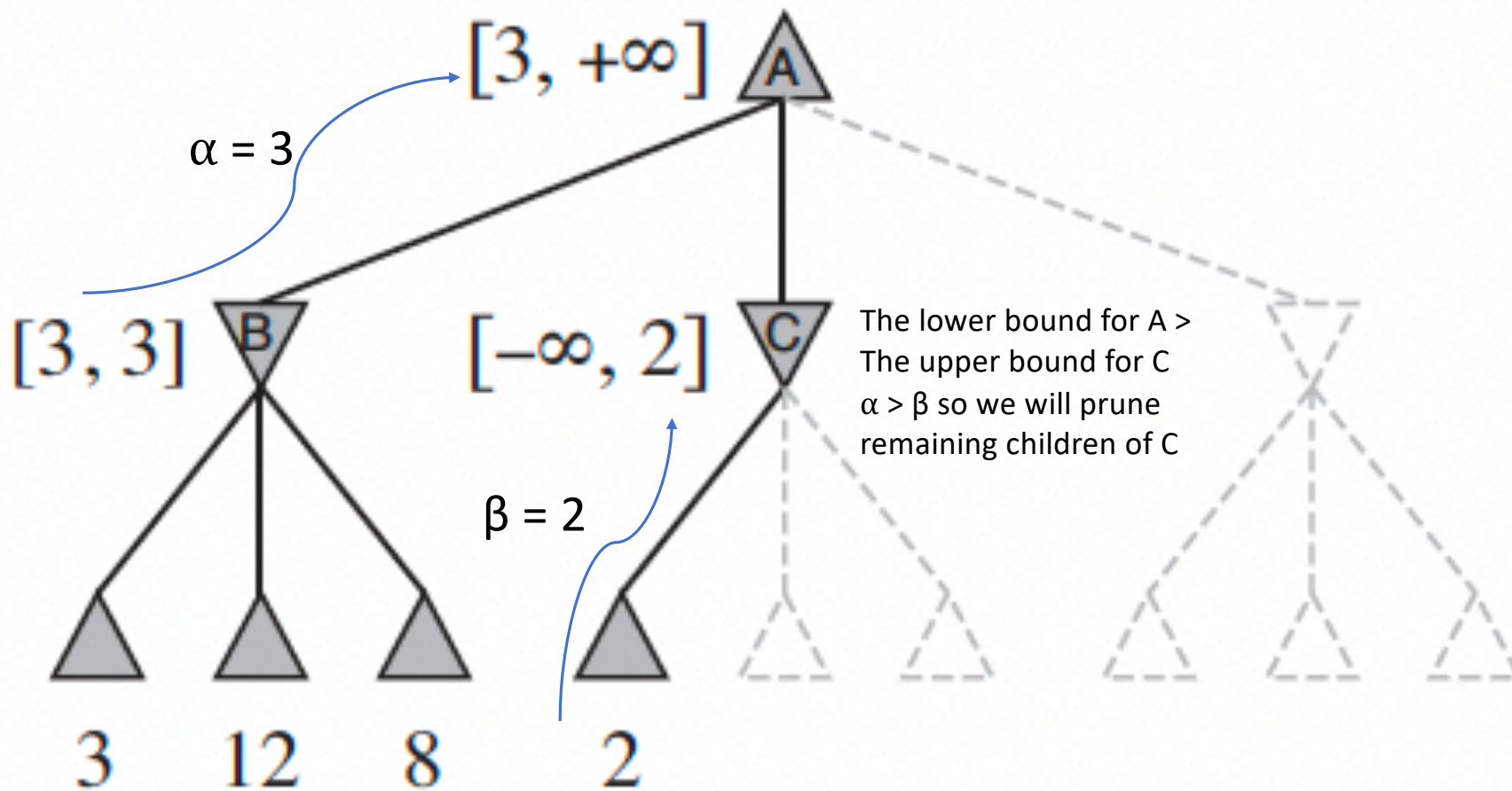
(b)



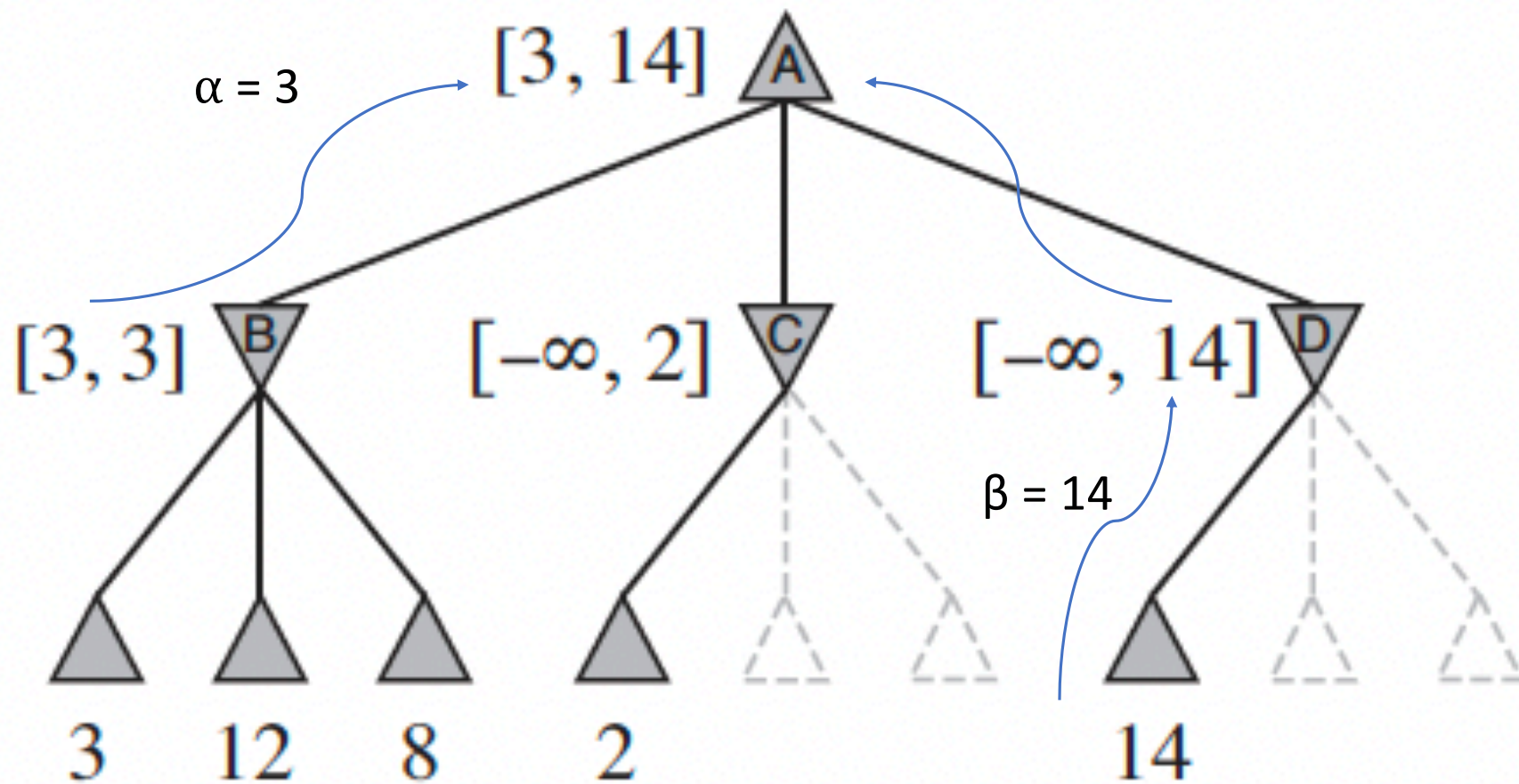
(c)



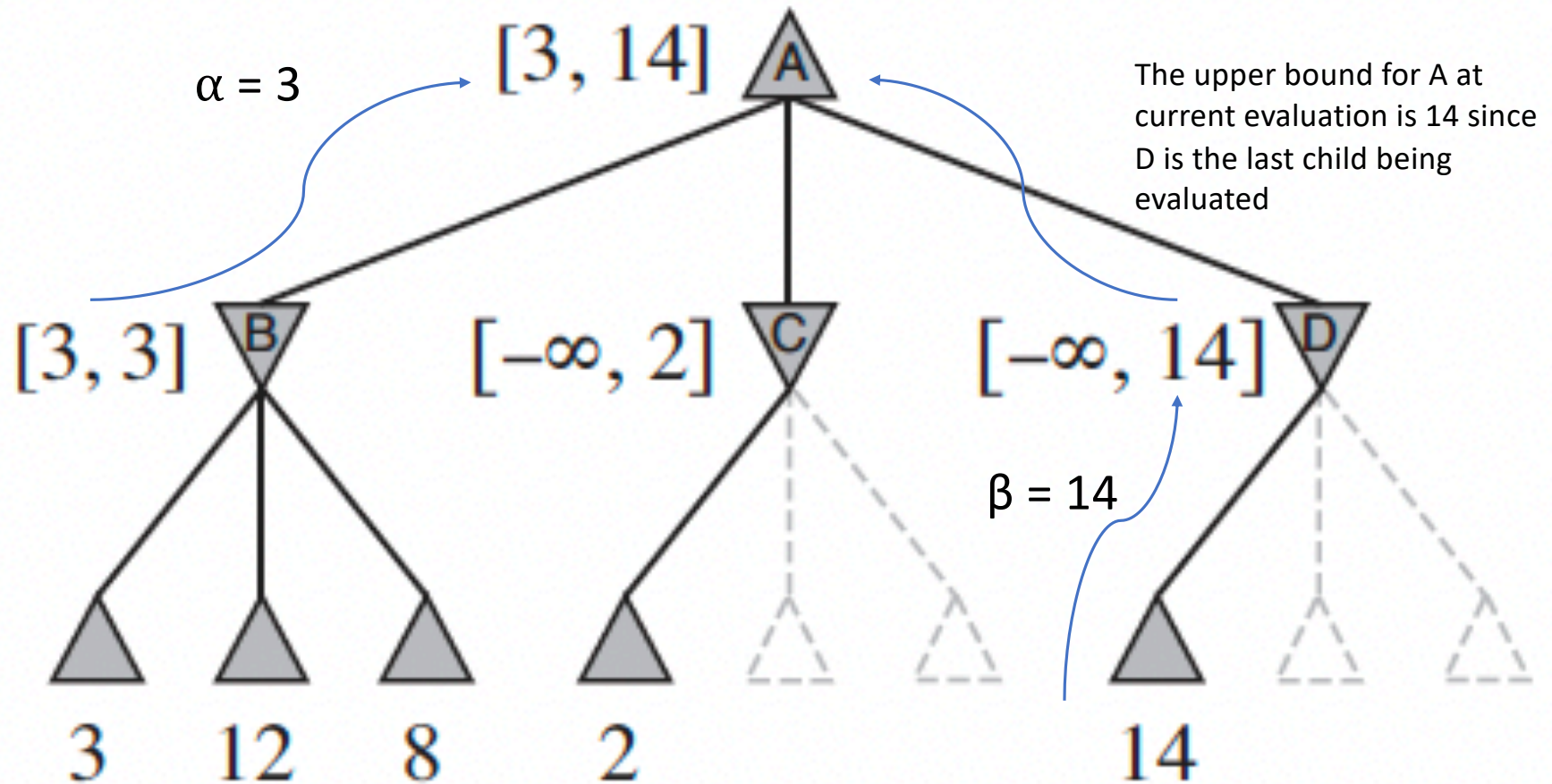
(d)



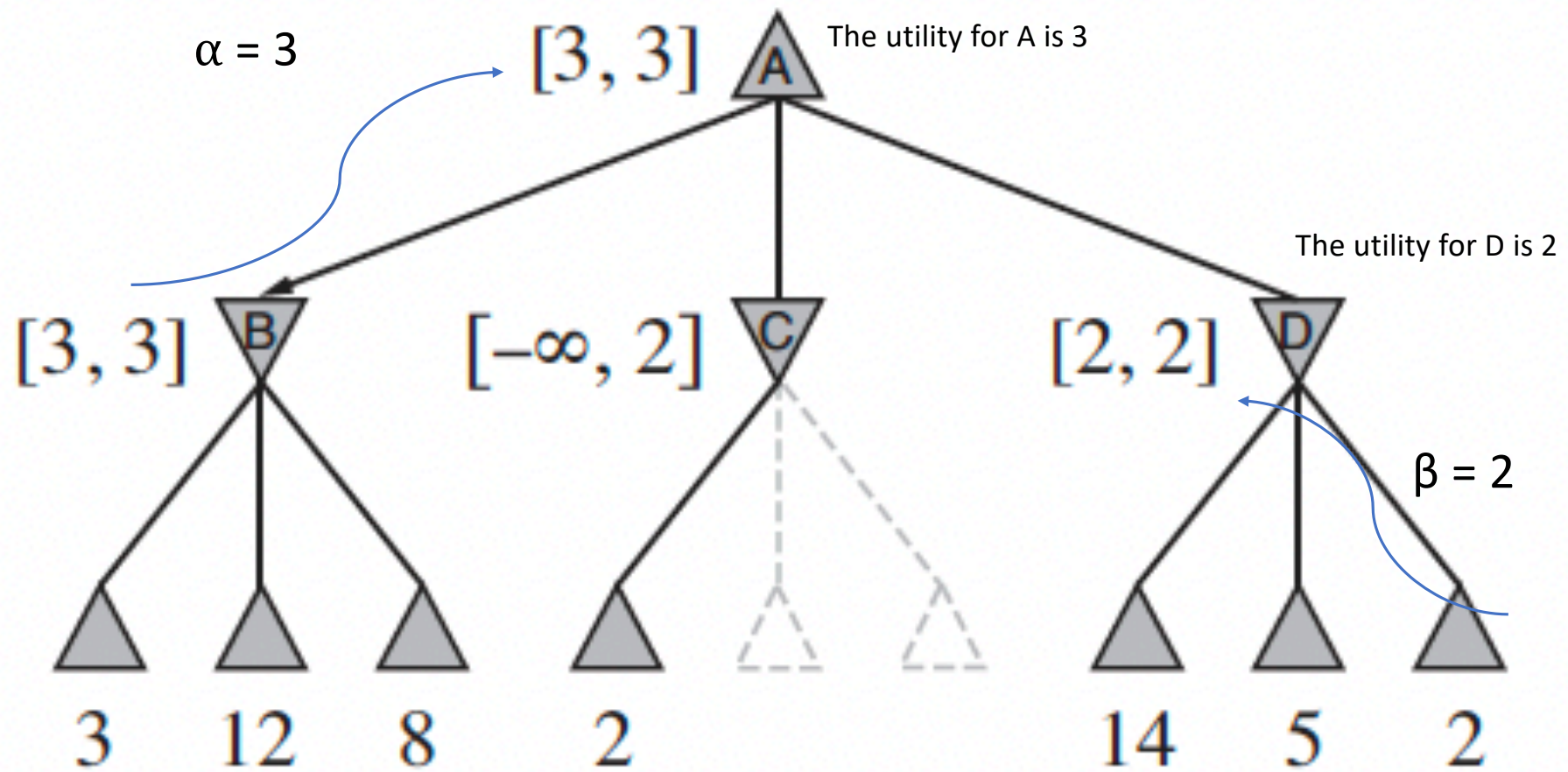
(e)



(e)



(f)



Activity – Alpha-Beta Pruning

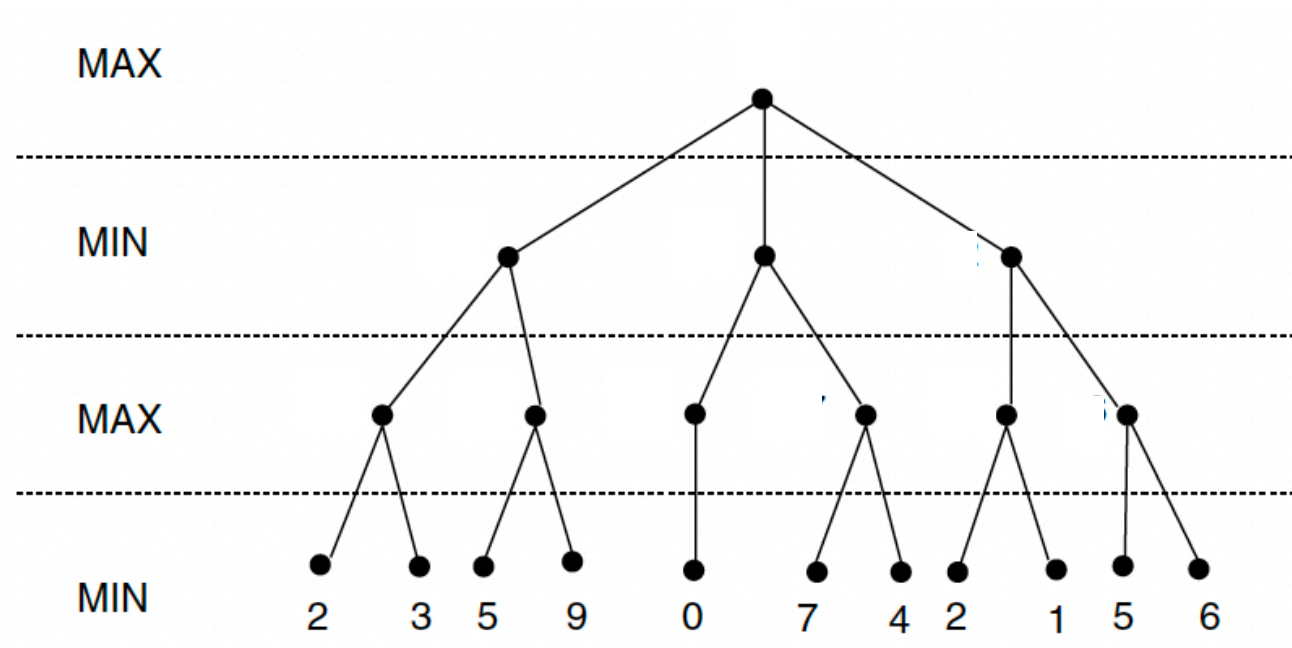
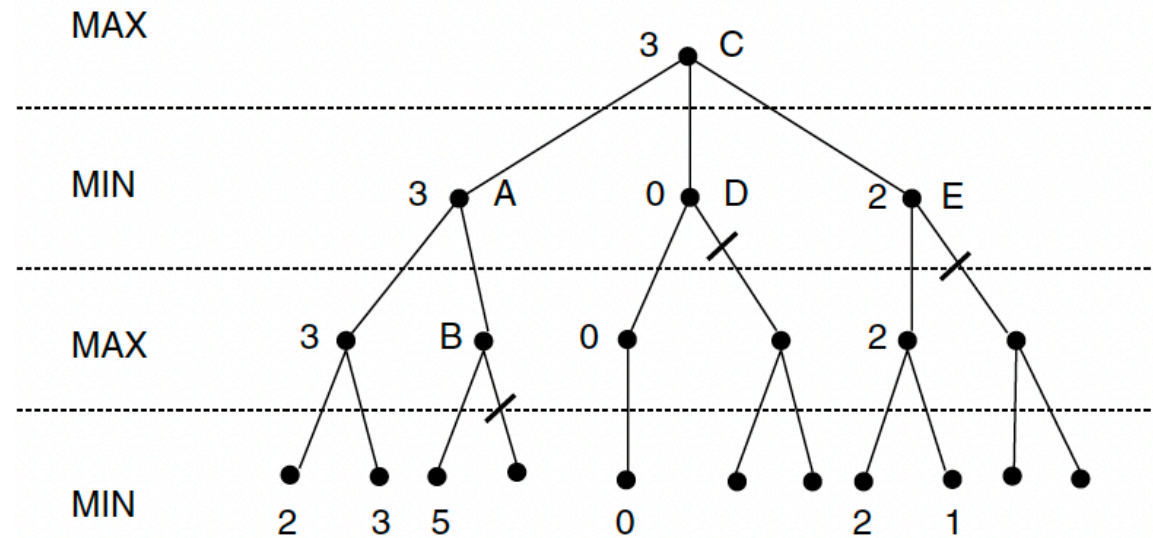


Figure 4.21 Minimax to a hypothetical state space. Leaf states show heuristic values; internal states show backed-up values.

Solution



A has $\beta = 3$ (A will be no larger than 3)
 B is β pruned, since $5 > 3$
 C has $\alpha = 3$ (C will be no smaller than 3)
 D is α pruned, since $0 < 3$
 E is α pruned, since $2 < 3$
 C is 3

Figure 4.26 Alpha-beta pruning applied to state space of Figure 4.21. States without numbers are not evaluated.